AWS Cloud Operations Blog

Best Practices for validating AWS AppConfig Feature Flags and Configuration Data

by Steve Rice and Chloe Goldstein | on 29 AUG 2022 | in Advanced (300), AWS Config, Best Practices, Customer Solutions, Management & Governance, Management Tools, Technical How-to | Permalink | Share

AWS AppConfig helps you create, manage, and deploy application configuration. One crucial use case for AppConfig is feature flagging, which lets you release features quickly and safely. Using AppConfig Feature Flags, you can separate code from configuration data and hide new features behind a configuration flag. When ready to release that feature, you simply update the flag value, rather than pushing out any new code. This practice is safe and effective since you can tune your application at runtime without deploying or restarting the app.

AppConfig and feature flagging help with CI/CD/CC processes, where CC represents Continuous Configuration. Continuous configuration lets you update dynamic configuration values during runtime, all without deploying new code or restarting the app. It's also the practice of rolling out those changes in a controlled and gradual way.

But with this great power comes great responsibility. Toggling on flags or releasing configuration data can be just as dangerous as pushing untested code to production. Errors caused by feature flag or configuration updates could happen anytime, causing anywhere from small to catastrophic damage. It's crucial that you only deploy valid configuration data. AppConfig has the tools you need to ensure that harmful configs aren't deployed.

Internally at Amazon, thousands of teams use AppConfig. One best practice we've learned and built into AppConfig is the ability to validate and check configurations before deploying configuration changes. You can use AppConfig Validators to make sure that your configuration data functions as intended.

Validators ensure that configuration deployments proceed only when they are valid, or as expected. Suppose your configuration contains Amazon Elastic Compute Cloud (Amazon EC2) information. In that case, you can ensure that your instance type metadata is accurate. Provisioning the wrong instance type could lead to performance issues or unintended costs. You can create Validators in either **JSON Schema** or as an **AWS Lambda function**. You can also create **Constraints** for Feature Flags which help you add restrictions to your Feature Flag data. These tools are crucial safety mechanisms that will help you create a robust production workload. The following are various best practices, use cases, and guidance for deploying valid configuration data.

It should be noted that another AWS AppConfig recommendation besides using Validators, is to use AWS AppConfig Agent to retrieve configuration data. The Agent provides built-in best practices to do caching and polling, which can greatly improve performance and availability for your application compared to calling the AWS AppConfig service directly. You can read more about using AWS AppConfig Agent here.

Validate your configurations to make sure that they're both functional and effective

It is a best practice to make sure that all of your configuration data has an associated Validator.

Specifically, you must verify that strings, booleans, numbers, and arrays are placed and written correctly in your configuration data. For example, you may use JSON schema to check that your configuration contains a boolean

statement where necessary; the deployment of new configuration data will fail if a number or string are found instead.

You also want to consider external factors that are associated with your configuration data. Although your configuration might not have a syntax error, it may contain a value that could be problematic if deployed. For example, you may need to confirm that your server's configuration has specific port numbers open and available. This is not a syntax validation, but a functional semantic validation.

AppConfig's Lambda Validators provide great flexibility because you can write code to ensure that your configurations work as intended. Per the example above, you could write a semantic Lambda Validator to connect through the port that is used in the configuration data prior to pushing that change across your entire fleet.

Creating Validators is straightforward. In the following screenshot of the AWS Management Console, you can see that you're prompted to create JSON Schema and Lambda Validators after you create a configuration profile.

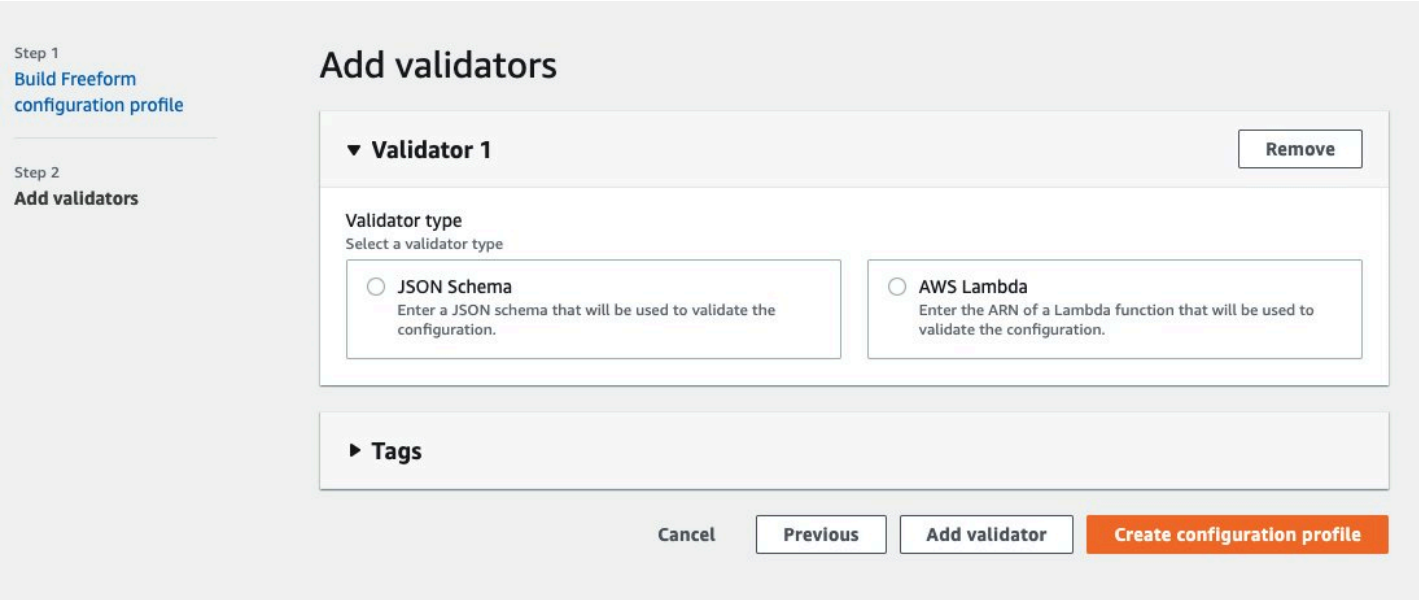

Figure 1. Console Screenshot showing options to add validators.

Make Validators as restrictive as reasonably possible

When authoring a Validator, it's important that you account for any possible configuration issue. Make sure that anything outside of expected values, especially the extremes, are rejected by your Validator. For example, when writing JSON schema, you'll want to confirm that numbers have minimum and maximum values defined to avoid extreme values from being deployed, including negative numbers.

Although it's critical that you limit extreme values, you don't want to be so restrictive that you block a legitimate configuration update. For example, suppose you have an e-commerce website with a Feature Flag that throttles the number of results returned in pagination. In that case, you can be more flexible with the maximum value, assuming that your database can handle it.

Use Validators to inform and standardize your configurations

By authoring Validators and determining how data must be structured and defined, you'll establish standards and rules for your configurations. For example, suppose your application can only handle names with less than 40 characters. In that case, this constraint will likely become apparent as you craft Validators. By realizing restrictions sooner rather than later, you can avoid any unexpected issues or outages. Furthermore, you can share these insights across your organization. This practice will set standards across your teams and will enable everyone to better understand your application performance.

When creating AppConfig Feature Flags, use Constraints to make sure that unexpected values aren't deployed to your application

When using Feature Flags, **Attributes** let you add additional values within your Flag. You can also validate attribute values against specified **Constraints** to ensure that any unexpected values aren't deployed to your application. For instance, suppose you have a feature flag to turn on discounts for an e-commerce website. You can create a feature flag with a number as the value to indicate the percentage for a sale. In this case, you would want to have minimum and maximum values, such as 10 and 90 as your constraint values. Constraints provide an additional safety guard because you won't be able to input an extreme value like 180. AppConfig currently supports constraints for strings, numbers, booleans, string arrays, and number arrays.

In this screenshot, you can see this sample constraint implemented.

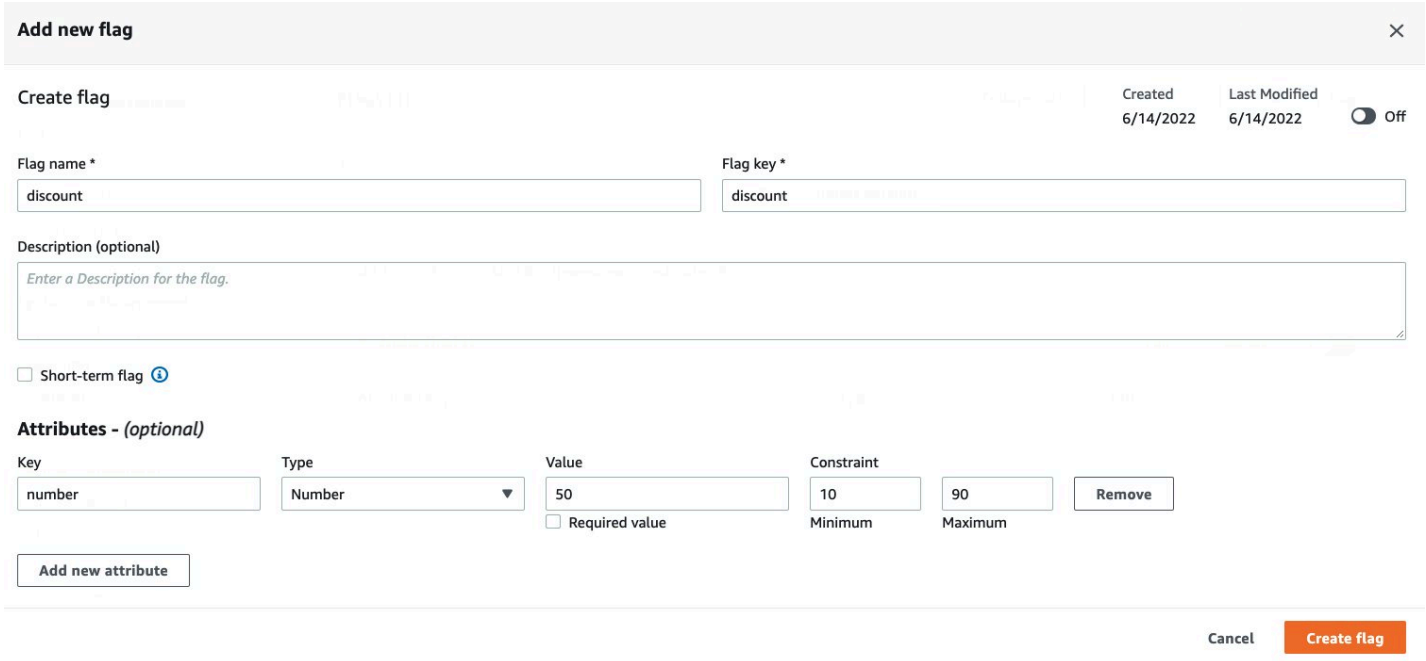

Figure 2. Console Screenshot of a Constraint

Keep your Validators aligned with updated Feature Flag data and Configuration Profiles

Validators should be stored and maintained appropriately with version control in place. If you want to add a new feature flag or have to update a configuration profile, then remember to account for those changes in your Validators. New configuration data or values may require new logic or syntactic checks. For example, if you use endpoint values as your configuration data, then you should add a Validator to check that the endpoint URL is correctly formatted and uses the correct protocol.

Use Validators with gradual deployments and automatic rollbacks.

In addition to Validators, gradual deployments and automatic rollbacks are great safety guard rails. With deployment strategies, AppConfig lets you deploy your configurations and Feature Flags gradually so that you limit the impact of any unintended outcome. Deployment Strategies have different deployment types, deployment times, and bake time. **Deployment time** refers to the amount of time AppConfig deploys to hosts. **Bake time** specifies the amount of time that AppConfig monitors for **Amazon CloudWatch alarms** before proceeding to the next step of a deployment or before considering the deployment to be complete. If an alarm is triggered during this time, AppConfig automatically rolls back the deployment. These automatic rollbacks mean that you can add another layer of security to your workload. Even if your configuration data is valid, but the deployment is still causing a 5xx error generated by your application, AppConfig will roll back the deployment.

As shown in the following, you can select a gradual method for your deployment strategy before starting your deployment.

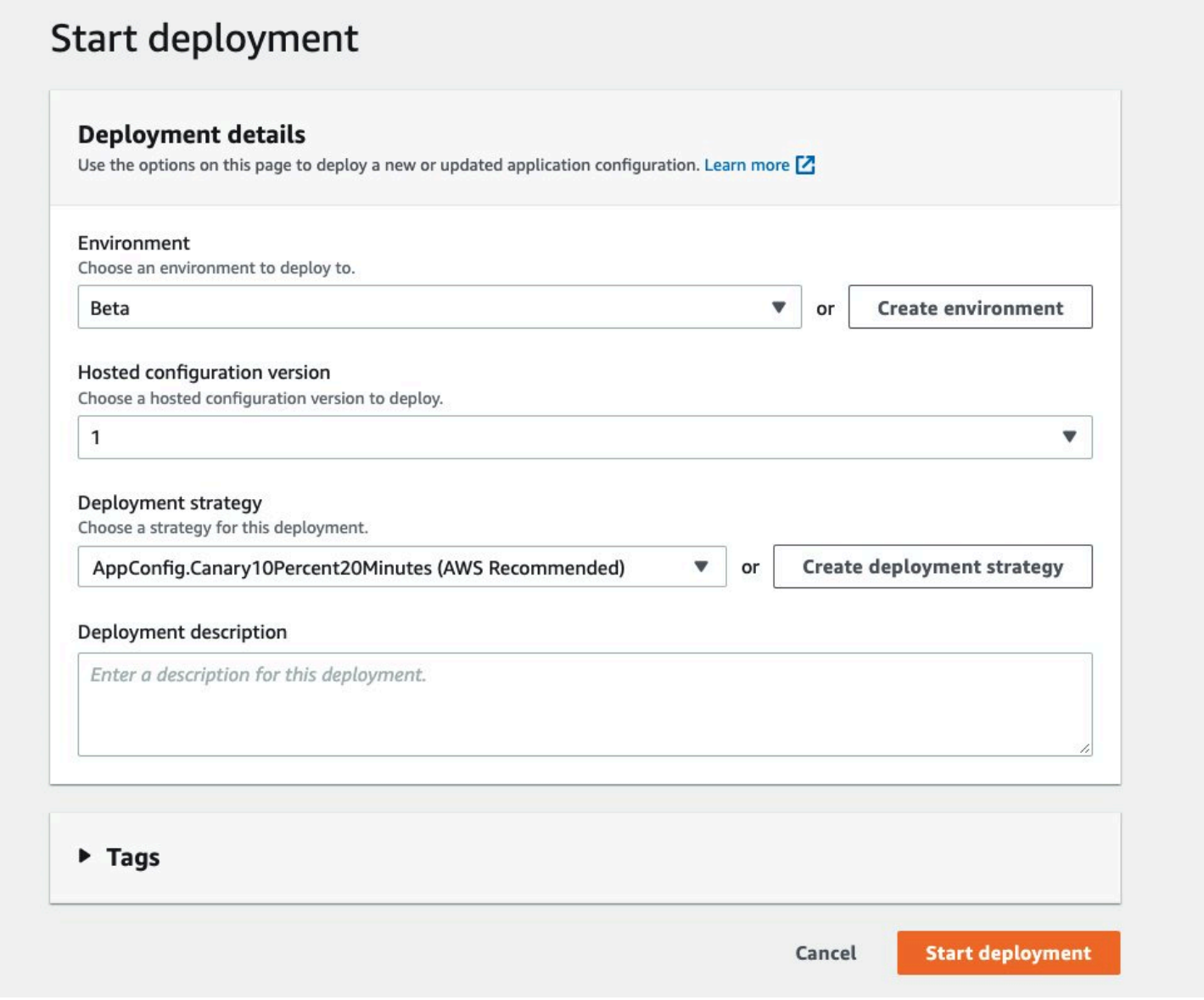

Figure 3. Console Screenshot of setting up a gradual deployment.

Create a runbook for a failed deployment

If a Validator identifies an error with your configuration, make sure that your teams are ready to handle the failed deployment appropriately. Although no problematic code will be pushed to production, you must troubleshoot the configuration error. You want to understand the implications and risks involved with a failed deployment, such as resource dependencies, coordination between teams, necessary escalation, audit trails, and scheduling. One key solution to account for these risks is a runbook solution that outlines the necessary remediation steps. You can learn more about runbooks and security incident response guides here.

Conclusion

This post showed you how to use AppConfig optimally for safe development processes. When using AppConfig, it's critical that you use Validators and Constraints to make sure that your configuration is functional and practical. These restrictions should be as limiting as reasonably possible. By creating Validators, you can additionally standardize practices across teams and gain insights on your application. These controls can be enhanced with gradual deployments, automatic rollbacks, and a runbook to handle any failed deployments. With these best practices in mind, your teams can successfully validate configurations configuration data in AppConfig.

Get Started!

To get started with AppConfig today, you can set up your Feature Flags and configuration data in the AWS AppConfig Management Console. You can also visit the AppConfig Product Page or the AppConfig Documentation to learn more.

About the Author:

TAGS: AWS AppConfig, DevOps, Dynamic Configuration, feature flags, Systems Manager